

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 02 · LECTURE NOTES

Search

One algorithm, five strategies.

MODULE OUTCOMES

Formulate a problem as a state space.

State the one generic search algorithm and what varies in it.

Compare the uninformed strategies on four criteria.

Explain why graph search needs a closed set and tree search does not.

Estimate whether a state space is searchable at all.

1 Formulating a search problem

A SEARCH PROBLEM IS FIVE THINGS, AND ONE OF THEM IS A DESIGN DECISION

An initial state; the set of actions available in each state; a transition function giving the resulting state; a goal test; and a cost for each action. That is the complete specification, and every problem in Modules 02 through 07 is written in this form.

The choice of what counts as a state is the consequential decision, and it is yours — which is exactly CSCE 669 Module 12 §1's decision-variable choice arriving in a different subject, with the same property that it determines tractability more than any later optimisation.

Is a state a grid cell? A grid cell plus a facing direction? Plus which keys you are carrying? Plus the positions of movable crates? Each gives a different state space size and a different set of problems that are solvable at all.

Too coarse and the solution is wrong — a path that ignores which doors you have keys for is not a path. **Too fine and the space is unsearchable,** because every added component multiplies the size. **Get this right before writing any code,** and write the state space size down as a number.

```

frontier = { start }
explored = { }

while frontier not empty:
    node = frontier.REMOVE()      # <-- the only difference
    if goal(node): return path(node)
    explored.add(node.state)
    for each successor s of node:
        if s.state not in explored and s not in frontier:
            frontier.add(s)
return failure

```

THE ONLY THING THAT VARIES BETWEEN ALGORITHMS IS WHICH NODE REMOVE() RETURNS:

a FIFO queue	-> breadth-first
a LIFO stack	-> depth-first
a priority queue on $g(n)$	-> uniform cost
a priority queue on $h(n)$	-> greedy best-first
a priority queue on $g(n)+h(n)$	-> A* (Module 03)

THAT IS THE WHOLE TAXONOMY. Every search algorithm in Modules 02 through 07 is this loop with a different frontier discipline and a different state space.

THE EXPLORED SET is what makes this GRAPH search. Without it you have TREE search, which revisits states and can loop forever in a cyclic space. With it, memory grows with the number of distinct states reached (section 3).

Writing this loop once and parameterising the frontier is the right implementation decision — it makes the comparisons of §2 a one-line change rather than five separate programs, and it makes the unification visible in the code rather than only in the prose.

2 The uninformed strategies

Strategy	Complete?	Optimal?	Space
Breadth-first	Yes.	Only if all step costs are equal.	$O(b^d)$ — and this is usually the binding constraint, not the time (see the callout).
Depth-first	No — it can descend an infinite branch forever.	No.	$O(bm)$ — linear in depth. This is its only virtue, and it is a substantial one.
Uniform cost	Yes.	Yes — for any non-negative costs.	$O(b^{C^*/\epsilon})$, which can exceed breadth-first when there are many cheap steps.
Iterative deepening	Yes.	If costs are equal.	$O(bd)$ — breadth-first's guarantees at depth-first's memory. See the footnote.

Strategy	Complete?	Optimal?	Space
Bidirectional	Yes.	If costs are equal.	$O(b^{d/2})$ — a square root, which is the largest asymptotic win available without a heuristic. Requires a predecessor function and a way to test frontier intersection.

Iterative deepening is the right default when no heuristic is available, and the reason is worth working through because it is counterintuitive: it re-searches the top of the tree at every iteration, which sounds wasteful. **But the number of nodes at depth d is b^d , and the sum of all shallower levels is about $b^d/(b-1)$ — so the repeated work is a constant factor, around 11% at $b = 10$. The deepest level dominates the node count, so repeating everything above it is nearly free.**

MEMORY, NOT TIME, IS WHAT STOPS A SEARCH

Breadth-first search at branching factor 10 and depth 12 stores on the order of 10^{12} nodes. At even 100 bytes per node that is 100 terabytes of frontier.

The time would be tolerable on a cluster; the memory is not available anywhere. So the frontier size is the binding constraint in practice, and it is the first quantity to estimate — which is the opposite of the intuition most people bring, that search is slow.

Which is why iterative deepening exists at all (it trades time for memory, in the favourable direction), and why memory-bounded heuristic variants such as IDA* and SMA* exist for the informed case (Module 03 §4).

So estimate b and d before implementing anything. b^d tells you immediately whether the problem needs a heuristic, a reformulation (§4), or an approximation — and the estimate takes two minutes while discovering it empirically takes a day.

3 Graph search against tree search

- **In a cyclic state space, tree search can loop forever** — walking back and forth between two states, each time generating the other as a fresh successor. The search is not slow; it does not terminate.
- **In an acyclic space it still explodes.** A grid with four movement actions has exponentially many distinct paths to each cell, and **tree search explores all of them**, although they all arrive at the same place.
- **So the explored set converts an exponential node count into one polynomial in the number of states** — which is frequently the difference between a search that finishes and one that does not, and is a far larger effect than any constant-factor optimisation.
- **The cost is memory proportional to the number of states reached**, which is **why depth-first search's linear memory advantage is lost** as soon as you add duplicate detection — a trade worth making explicitly rather than by default.
- **And the state must be hashable and compared correctly. A subtle equality bug here produces a search that is merely slow rather than broken** — duplicates are not detected, the node count inflates, and **nothing alerts you because the answers are still correct. Instrument the duplicate-detection rate** and look at it: if almost nothing is detected as a duplicate, either your state representation is too fine (§1) or your equality is wrong.

4 Reformulation

SHRINKING THE STATE SPACE BEATS SPEEDING UP THE SEARCH

Abstraction. Search over rooms rather than individual tiles, find a room-level path, then refine within each room. **This is hierarchical pathfinding** (Module 04 §3), it reduces the effective depth dramatically, and it is the single most used reformulation in games.

Symmetry elimination. If two states are equivalent under a symmetry of the problem, search only one of them. **This is CSCE 669 Module 09 §4's symmetry breaking** arriving in a different subject, with the same enormous effect — and the same property of being invisible until you look for it.

Macro-actions. Collapse a commonly useful action sequence into a single action, reducing depth at the cost of increasing the branching factor. **Since depth is in the exponent and branching is the base, this is usually favourable**, which is the same arithmetic as bidirectional search's square root.

And a better state encoding. Dropping a state component that the solution does not depend on divides the space by that component's range — so **ask of every state component whether any solution's correctness depends on it**, and remove the ones that fail. This is the cheapest and most frequently available win, and §1's warning applies in the other direction too.

WHERE THIS GOES NEXT

Module 03 adds a heuristic, which is the single largest available improvement — **it changes the effective branching factor rather than a constant**, so the effect compounds with depth.

Module 04 applies it to the track's problem, pathfinding, with the engineering that production systems require and that the algorithm alone does not supply.

Module 05 searches against an opponent, where the successor function alternates whose choice it is, and the evaluation becomes a minimax rather than a cost.

Modules 06 and 07 exploit problem structure — constraint networks and action preconditions — **to prune the frontier and to derive heuristics automatically rather than by hand. All of it is the loop in §1**, which is worth keeping in mind as the state spaces get stranger.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapter 3 (pseudocode free)	The reference for this module. The generic algorithm of §1 and the comparison table of §2 are from here.
Berkeley CS188 — search lectures and Project 1 (free)	The Pacman search project , which is the best available exercise for this material and includes the state-formulation decisions of §1.
Korf — Depth-First Iterative-Deepening (free)	The §2 result , with the node-count argument worked out properly.
Red Blob Games — Introduction to Graph Search (free, interactive)	Interactive visualisations of every strategy in §2 , and the clearest intuition-builder available.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement the generic search loop once** and parameterise the frontier. Instantiate all five strategies.
- 2 **Formulate one problem three ways** with different state definitions, and report the state space size of each.
- 3 **Report nodes expanded for each strategy** on the same problem.
- 4 **Confirm iterative deepening's overhead** empirically and compare against the predicted constant factor.
- 5 **Run tree search on a cyclic space** and confirm it does not terminate.
- 6 **Instrument the duplicate-detection rate** and report it for two state representations.

- 7 **Introduce an equality bug deliberately** and measure the node count inflation. Confirm the answers stay correct.
- 8 **Estimate b and d** for a problem you care about, and compute b^d .
- 9 **Implement bidirectional search** and verify the square-root node count.
- 10 **Apply one reformulation from §4** and report the state space reduction.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the five components of a search problem.
- 2 Why is the state definition the consequential decision?
- 3 Write the generic search loop and say what varies between algorithms.
- 4 Compare five strategies on completeness, optimality, and space.
- 5 Why is iterative deepening's overhead only a constant factor?
- 6 Why is memory rather than time the binding constraint?
- 7 What does the explored set buy and cost?
- 8 Why is an equality bug in the state particularly dangerous?
- 9 Name four reformulations and what each reduces.